Kubernetes Architecture - Interview Questions & Answers

Q1. What are the main components of Kubernetes Architecture?

The Kubernetes architecture is divided into two main sections:

1. Control Plane (Master Node)

The Control Plane is the brain of Kubernetes. It is responsible for making cluster-wide decisions, maintaining the desired state, scheduling workloads, and monitoring the overall health of the cluster.

Components:

- kube-apiserver
- etcd
- kube-scheduler
- kube-controller-manager

2. Worker Nodes

Worker Nodes are responsible for running the actual application workloads in the form of Pods.

Components:

- kubelet
- kube-proxy
- Container Runtime

Q2. How do the Control Plane and Worker Nodes communicate?

All communication in Kubernetes happens through the **kube-apiserver**.

Worker node components such as kubelet communicate securely with the API Server using TLS encryption.

Kubernetes components do not communicate directly with each other. Instead, they read from and write to the API Server, making it the central communication hub of the cluster.

Q3. What is the role of kube-apiserver?

The kube-apiserver is the central management component and the entry point of the Kubernetes Control Plane.

Responsibilities

- Exposes the Kubernetes REST API
- Receives requests from users and cluster components
- Authenticates and authorizes requests
- Validates resource definitions
- Stores and retrieves cluster state from etcd
- Acts as the communication hub for all Kubernetes components

Since every operation passes through the API Server, it is considered the front door of Kubernetes.

Q4. What is etcd and why is it critical?

etcd is a distributed and highly available key-value database used by Kubernetes.

It serves as the **single source of truth** for the entire cluster.

etcd Stores

- Pod information
- Deployment configurations
- Services
- Secrets
- ConfigMaps
- Node information
- Cluster metadata

If etcd becomes unavailable or its data is corrupted, the cluster loses its state information, making etcd backups extremely important.

Q5. How does the kube-scheduler choose a target Node for a Pod?

The kube-scheduler is responsible for assigning newly created Pods to suitable Worker Nodes.

Scheduling Process

1. Filtering Phase

The scheduler eliminates nodes that do not meet the Pod's requirements.

Examples:

- Insufficient CPU
- Insufficient Memory
- Node Selector mismatch
- Affinity/Anti-Affinity constraints
- Taints and Tolerations

2. Scoring Phase

The remaining nodes are scored based on scheduling policies.

The scheduler selects the node with the highest score and binds the Pod to that node.

Q6. What does the kube-controller-manager do?

The kube-controller-manager runs multiple controllers responsible for maintaining the desired state of the cluster.

Controllers continuously execute a reconciliation loop:

```
Desired State
      ↓
Compare
      ↓
Actual State
      ↓
Correct Differences
```

Common Controllers

Node Controller

Detects node failures.

ReplicaSet Controller

Ensures the required number of Pod replicas are running.

Deployment Controller

Manages rolling updates and rollbacks.

The Controller Manager ensures the cluster remains healthy and consistent.

Q7. What is the exact function of kubelet?

kubelet is the primary node agent running on every Worker Node.

Responsibilities

- Registers the node with the Kubernetes cluster
- Watches for Pod assignments from the API Server
- Ensures Pods are running as specified
- Monitors container health
- Reports node and Pod status back to the API Server

The kubelet directly interacts with the container runtime to create and manage containers.

Q8. What is kube-proxy and how does it route network traffic?

kube-proxy is the networking component that runs on every Worker Node.

Responsibilities

- Watches Service and Endpoint objects
- Maintains networking rules
- Routes traffic to backend Pods
- Enables Service-based communication
- Supports load balancing

Traffic Flow

```
Client Request
      ↓
Service
      ↓
kube-proxy
      ↓
Backend Pod
```

It typically uses:

- iptables
- IPVS

Q9. What is the difference between a Pod and a Container?

Container

A Container is a lightweight, isolated package that contains:

- Application code
- Runtime
- Libraries
- Dependencies

Pod

A Pod is the smallest deployable unit in Kubernetes.

A Pod can contain one or more containers that share:

- Network namespace
- Storage volumes
- IP address
- Lifecycle

Key Difference

Container	Pod
Runs application processes	Hosts one or more containers
Docker concept	Kubernetes concept
Individual runtime unit	Smallest deployable Kubernetes unit

Q10. Differentiate between Deployment, StatefulSet, and DaemonSet

Deployment

Used for stateless applications.

Features:

- Easy scaling
- Rolling updates
- Rollbacks
- No fixed Pod identity

Examples:

- Web applications
- APIs
- Frontend services

StatefulSet

Used for stateful applications.

Features:

- Stable Pod names
- Persistent storage
- Ordered deployment
- Ordered termination

Examples:

- Databases
- Kafka
- Elasticsearch

DaemonSet

Ensures one Pod runs on every node.

Features:

- Automatically deploys Pods on new nodes
- One Pod per node

Examples:

- Log collectors
- Monitoring agents
- Security agents

Q11. Walk through the exact workflow when you run:

```
kubectl apply -f deployment.yaml
```

Step 1

The kubectl client sends the deployment configuration to the kube-apiserver.

Step 2

The API Server:

- Authenticates the request
- Validates the YAML definition
- Stores the configuration in etcd

Step 3

The Deployment Controller detects the new Deployment and creates a ReplicaSet.

Step 4

The ReplicaSet creates the required Pod objects.

Step 5

The Scheduler identifies Pods that do not have an assigned node.

Step 6

The Scheduler selects the most suitable Worker Node and records the assignment.

Step 7

The kubelet running on the selected node notices the assignment.

Step 8

The kubelet instructs the Container Runtime to:

- Pull the container image
- Create containers
- Start containers

Step 9

The Pod becomes operational and reports its status back to the API Server.

Complete Flow

```
kubectl
    ↓
API Server
    ↓
etcd
    ↓
Deployment Controller
    ↓
ReplicaSet
    ↓
Scheduler
    ↓
Worker Node
```

```
    ↓
Kubelet
    ↓
Container Runtime
    ↓
Pod Running
```

Q12. How does Kubernetes handle a Worker Node crash?

Step 1

The kube-controller-manager stops receiving heartbeats from the failed node.

Step 2

The Node Controller marks the node as unavailable or NotReady.

Step 3

Kubernetes identifies the Pods that were running on the failed node.

Step 4

The ReplicaSet or Deployment Controller detects that the actual number of running Pods is lower than the desired count.

Step 5

Replacement Pods are scheduled on healthy Worker Nodes.

Step 6

The kubelet on those healthy nodes starts the new Pods.

Result

The application continues running despite the node failure.

This behavior is known as **Fault Tolerance** and is one of the key benefits of Kubernetes.